

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I, Nethra P(192511012) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nethra P

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SIMATS ENGINEERING

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python (scikit-learn).

(a) Load dataset, pre-process, and split into train/test sets

The Iris dataset (150 samples, 4 numeric features, 3 classes) was loaded via scikit-learn. It was checked for missing values (none found) and split into 70% training / 30% testing sets using stratified sampling.

(b) Build Decision Tree model (Gini Index) and identify root node

A DecisionTreeClassifier was trained using the Gini Index as the splitting criterion. The full tree structure was printed and the root node was identified as the feature giving the highest information gain (largest impurity reduction).

(c) Evaluate the model

The trained model was evaluated on the test set using accuracy score, a confusion matrix, and a classification report (precision, recall, F1-score). Misclassified instances were listed and the results were interpreted.

Code (Experiment 1)



```
1# Experiment 1: Decision Tree Classification
2# Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset
3
4import pandas as pd
5import numpy as np
6from sklearn.datasets import load_iris
7from sklearn.model_selection import train_test_split
8from sklearn.tree import DecisionTreeClassifier, export_text
9from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
10
11# ----- (a) Load, pre-process, split -----
12iris = load_iris()
13df = pd.DataFrame(iris.data, columns=iris.feature_names)
14df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)
15
16print("First 5 rows of the dataset:")
17print(df.head(), "\n")
18
19print("Data types:")
20print(df.dtypes, "\n")
21
22print("Missing values per column:")
23print(df.isnull().sum(), "\n")
24# No missing values in Iris; encoding not required for features (all numeric).
25# Target is label-encoded already via iris.target (0=setosa,1=versicolor,2=virginica)
26
27X = iris.data
28y = iris.target
29
30X_train, X_test, y_train, y_test = train_test_split(
31    X, y, test_size=0.3, random_state=42, stratify=y
32)
33print(f"Training set size: {X_train.shape[0]} samples")
34print(f"Testing set size : {X_test.shape[0]} samples\n")
35
36# ----- (b) Build Decision Tree (Gini Index) -----
37clf = DecisionTreeClassifier(criterion='gini', random_state=42)
38clf.fit(X_train, y_train)
39
40print("Decision Tree structure:")
```

```
exp1_decision_tree.py - Online Python Compiler (part 2/3)

41tree_rules = export_text(clf, feature_names=list(iris.feature_names))
42print(tree_rules)
43
44root_feature_index = clf.tree_.feature[0]
45root_threshold = clf.tree_.threshold[0]
46root_feature_name = iris.feature_names[root_feature_index]
47print(f"Root Node: '{root_feature_name}' ≤ {root_threshold:.2f}")
48print("Justification: The root node is chosen because it has the HIGHEST information gain")
49print("(largest reduction in Gini impurity) among all features at the top of the tree -")
50print(f"'{root_feature_name}' best separates the three Iris species in a single split.\n")
51
52# ----- (c) Evaluate the model -----
53y_pred = clf.predict(X_test)
54
55acc = accuracy_score(y_test, y_pred)
56print(f"Accuracy Score: {acc:.4f}\n")
57
58cm = confusion_matrix(y_test, y_pred)
59print("Confusion Matrix:")
60print(cm, "\n")
61
62print("Classification Report:")
63print(classification_report(y_test, y_pred, target_names=iris.target_names))
64
65# Misclassified instances
66misclassified_idx = np.where(y_pred ≠ y_test)[0]
67print(f"Number of misclassified instances: {len(misclassified_idx)}")
68if len(misclassified_idx) > 0:
69    print("Misclassified instances (Actual → Predicted):")
70    for i in misclassified_idx[:5]:
71        actual = iris.target_names[y_test[i]]
72        predicted = iris.target_names[y_pred[i]]
73        print(f"Sample {i}: Features={X_test[i]}, Actual={actual}, Predicted={predicted}")
74else:
75    print("No misclassified instances in this run; example of a borderline versicolor/")
76    print("virginica sample (typically confused due to overlapping petal measurements).")
77
78print("\nComment: The model performs very well on Iris because the three classes are")
79print("almost linearly separable using petal length/width. Most (if not all) errors,")
80print("when present, occur between 'versicolor' and 'virginica' since their petal")
```

```
exp1_decision_tree.py - Online Python Compiler (part 3/3)

81print("measurements overlap, whereas 'setosa' is always perfectly classified.")
```

Output (Experiment 1)

```
Output - Experiment 1 (part 1/3)

First 5 rows of the dataset:
  sepal length (cm)  sepal width (cm)  ...  petal width (cm)  species
0                5.1                3.5  ...                0.2    setosa
1                4.9                3.0  ...                0.2    setosa
2                4.7                3.2  ...                0.2    setosa
3                4.6                3.1  ...                0.2    setosa
4                5.0                3.6  ...                0.2    setosa

[5 rows x 5 columns]

Data types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species              category
dtype: object

Missing values per column:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
dtype: int64

Training set size: 105 samples
Testing set size : 45 samples

Decision Tree structure:
|--- petal length (cm) ≤ 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) ≤ 1.55
|   |   |--- petal length (cm) ≤ 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.55
|   |   |   |--- petal width (cm) ≤ 1.70
```

```
Output - Experiment 1 (part 2/3)

| | | |--- sepal width (cm) ≤ 2.85
| | | | |--- class: 1
| | | |--- sepal width (cm) > 2.85
| | | | |--- class: 2
| | | |--- petal width (cm) > 1.70
| | | |--- petal length (cm) ≤ 4.85
| | | | |--- sepal width (cm) ≤ 3.00
| | | | | |--- class: 2
| | | | |--- sepal width (cm) > 3.00
| | | | | |--- class: 1
| | | |--- petal length (cm) > 4.85
| | | | |--- class: 2

Root Node: 'petal length (cm)' ≤ 2.45
Justification: The root node is chosen because it has the HIGHEST information gain
(largest reduction in Gini impurity) among all features at the top of the tree -
'petal length (cm)' best separates the three Iris species in a single split.

Accuracy Score: 0.9333

Confusion Matrix:
[[15  0  0]
 [ 0 12  3]
 [ 0  0 15]]

Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00        15
  versicolor         1.00        0.80        0.89        15
   virginica         0.83        1.00        0.91        15

   accuracy              0.93        45
  macro avg         0.94        0.93        0.93        45
 weighted avg         0.94        0.93        0.93        45

Number of misclassified instances: 3
Misclassified instances (Actual → Predicted):
Sample 3: Features=[6.3 3.3 4.7 1.6], Actual=versicolor, Predicted=virginica
Sample 32: Features=[6.  3.4 4.5 1.6], Actual=versicolor, Predicted=virginica
```

```
Output - Experiment 1 (part 3/3)

Sample 39: Features=[6.7 3.  5.  1.7], Actual=versicolor, Predicted=virginica

Comment: The model performs very well on Iris because the three classes are
almost linearly separable using petal length/width. Most (if not all) errors,
when present, occur between 'versicolor' and 'virginica' since their petal
measurements overlap, whereas 'setosa' is always perfectly classified.
```

Result Summary — Experiment 1

- Root node: 'petal length (cm)' ≤ 2.45 — chosen because it yields the highest information gain / largest Gini impurity reduction among all features.
- Test accuracy: 93.33% (42 correct out of 45 test samples).
- Confusion matrix shows all 3 misclassifications occurred between 'versicolor' and 'virginica' (predicted as virginica), while 'setosa' was classified with 100% accuracy — consistent with the known overlap between versicolor and virginica petal measurements.

- Example misclassified instance: Sample 3 — Features=[6.3, 3.3, 4.7, 1.6], Actual=versicolor, Predicted=virginica.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4×4 Grid World environment and observe the agent's learned policy.

(a) Define the environment

A 4×4 grid (16 states, numbered 0–15) was defined with 4 actions (Up, Down, Left, Right). The goal state is cell 15 (bottom-right, reward +10), obstacle states are cells 5, 7, and 11 (reward −5), and every other step gives reward −1. The Q-table (16×4) was initialised with zeros. Hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\epsilon = 0.2$ (ϵ -greedy policy).

(b) Run Q-Learning for 100 episodes

The agent was trained for 100 episodes starting from state 0 each time, updating Q-values via the Bellman equation: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max(Q(s',\cdot)) - Q(s,a)]$. Q-values for two representative states (state 0 — start corner, and state 10 — mid-grid) were recorded at episodes 1, 50, and 100 to show how the table evolves from all-zero toward values that favour the goal-reaching actions.

(c) Final policy and convergence

The final greedy policy (best action per state) was extracted from the converged Q-table and displayed as a grid. The cumulative reward per episode was plotted, along with a 10-episode moving average, to show convergence behaviour.

Code (Experiment 2)

```
1# Experiment 2: Reinforcement Learning - Q-Learning on a 4x4 Grid World
2import numpy as np
3import matplotlib
4matplotlib.use('Agg')
5import matplotlib.pyplot as plt
6
7np.random.seed(42)
8
9# ----- (a) Define the environment -----
10GRID_SIZE = 4
11N_STATES = GRID_SIZE * GRID_SIZE
12ACTIONS = ['Up', 'Down', 'Left', 'Right']
13N_ACTIONS = len(ACTIONS)
14
15GOAL_STATE = 15 # bottom-right corner (row3,col3)
16OBSTACLE_STATES = [5, 7, 11] # a few blocked/penalty cells
17
18def state_to_rc(s):
19    return divmod(s, GRID_SIZE)
20
21def rc_to_state(r, c):
22    return r * GRID_SIZE + c
23
24def step(state, action):
25    r, c = state_to_rc(state)
26    if action == 'Up':
27        r = max(r - 1, 0)
28    elif action == 'Down':
29        r = min(r + 1, GRID_SIZE - 1)
30    elif action == 'Left':
31        c = max(c - 1, 0)
32    elif action == 'Right':
33        c = min(c + 1, GRID_SIZE - 1)
34    next_state = rc_to_state(r, c)
35
36    if next_state == GOAL_STATE:
37        reward = 10
38        done = True
39    elif next_state in OBSTACLE_STATES:
40        reward = -5
```

```
41     done = False
42     else:
43         reward = -1
44         done = False
45     return next_state, reward, done
46
47 # Q-table initialised with zeros
48 Q = np.zeros((N_STATES, N_ACTIONS))
49
50 # Hyperparameters
51 alpha = 0.1    # learning rate
52 gamma = 0.9    # discount factor
53 epsilon = 0.2  # exploration rate
54
55 print("Environment: 4x4 Grid World")
56 print(f"States: {N_STATES} Actions: {ACTIONS}")
57 print(f"Goal state: {GOAL_STATE} (reward +10) | Obstacle states: {OBSTACLE_STATES} (reward -5) | Step reward:
58 print(f"Hyperparameters → alpha (learning rate) = {alpha}, gamma (discount) = {gamma}, epsilon (exploration)
59
60 # ----- (b) Run Q-Learning -----
61 N_EPISODES = 100
62 MAX_STEPS = 100
63 rewards_per_episode = []
64
65 # States to track Q-value evolution
66 track_states = [0, 10]  # start corner, and a mid-grid state
67
68 q_history = {s: {} for s in track_states}
69
70 for episode in range(1, N_EPISODES + 1):
71     state = 0  # always start top-left corner
72     total_reward = 0
73
74     for t in range(MAX_STEPS):
75         if np.random.rand() < epsilon:
76             action_idx = np.random.randint(N_ACTIONS)
77         else:
78             action_idx = np.argmax(Q[state])
79
80         action = ACTIONS[action_idx]
```

```

81     next_state, reward, done = step(state, action)
82
83     best_next = np.max(Q[next_state])
84     Q[state, action_idx] += alpha * (reward + gamma * best_next - Q[state, action_idx])
85
86     total_reward += reward
87     state = next_state
88     if done:
89         break
90
91     rewards_per_episode.append(total_reward)
92
93     if episode in (1, 50, 100):
94         for s in track_states:
95             q_history[s][episode] = Q[s].copy()
96
97 print("Q-value evolution for tracked states (Up, Down, Left, Right):\n")
98 for s in track_states:
99     print(f"State {s} (row {state_to_rc(s)[0]}, col {state_to_rc(s)[1]}):")
100    for ep in (1, 50, 100):
101        vals = q_history[s].get(ep, Q[s])
102        vals_str = ", ".join(f"{a}={v:.3f}" for a, v in zip(ACTIONS, vals))
103        print(f"    Episode {ep:3d}: {vals_str}")
104    print()
105
106 # ----- (c) Extract final policy & plot -----
107 policy = {}
108 for s in range(N_STATES):
109     if s == GOAL_STATE:
110         policy[s] = 'GOAL'
111     else:
112         policy[s] = ACTIONS[np.argmax(Q[s])]
113
114 print("Final learned policy (grid layout, best action per state):")
115 for r in range(GRID_SIZE):
116     row_actions = []
117     for c in range(GRID_SIZE):
118         s = rc_to_state(r, c)
119         if s == GOAL_STATE:
120             row_actions.append(" GOAL ")

```

```
exp2_qlearning.py - Online Python Compiler (part 4/4)

121     elif s in OBSTACLE_STATES:
122         row_actions.append(f"{{policy[s][:4]}}")
123     else:
124         row_actions.append(f" {{policy[s][:4]:<4} ")
125     print(" | ".join(row_actions))
126 print()
127
128 avg_last_10 = np.mean(rewards_per_episode[-10:])
129 avg_first_10 = np.mean(rewards_per_episode[:10])
130 print(f"Average cumulative reward - first 10 episodes : {avg_first_10:.2f}")
131 print(f"Average cumulative reward - last 10 episodes : {avg_last_10:.2f}")
132 print("Convergence: The rising trend (and reduced variance) in cumulative reward over")
133 print("episodes shows the agent is learning a better policy - it is reaching the goal")
134 print("with fewer steps and fewer obstacle penalties as Q-values converge toward their")
135 print("optimal values via the Bellman update.")
136
137 # Plot cumulative reward per episode
138 plt.figure(figsize=(8, 5))
139 plt.plot(range(1, N_EPISODES + 1), rewards_per_episode, color='#2E86AB', linewidth=1.5)
140 # moving average for trend clarity
141 window = 10
142 moving_avg = np.convolve(rewards_per_episode, np.ones(window)/window, mode='valid')
143 plt.plot(range(window, N_EPISODES + 1), moving_avg, color='#E63946', linewidth=2, label=f'{window}-episode m
144 plt.title('Q-Learning: Cumulative Reward per Episode (4x4 Grid World)')
145 plt.xlabel('Episode')
146 plt.ylabel('Cumulative Reward')
147 plt.legend()
148 plt.grid(alpha=0.3)
149 plt.tight_layout()
150 plt.savefig('/home/claude/work/exp2_reward_plot.png', dpi=150)
151 print("\n[Plot saved as exp2_reward_plot.png]")
```

Output (Experiment 2)

```
Output - Experiment 2

Environment: 4x4 Grid World
States: 16 Actions: ['Up', 'Down', 'Left', 'Right']
Goal state: 15 (reward +10) | Obstacle states: [5, 7, 11] (reward -5) | Step reward: -1
Hyperparameters → alpha (learning rate) = 0.1, gamma (discount) = 0.9, epsilon (exploration) = 0.2

Q-value evolution for tracked states (Up, Down, Left, Right):

State 0 (row 0, col 0):
Episode 1: Up=-0.297, Down=-0.288, Left=-0.271, Right=-0.200
Episode 50: Up=-2.124, Down=-1.985, Left=-2.362, Right=-2.118
Episode 100: Up=-1.952, Down=1.034, Left=-2.320, Right=-2.177

State 10 (row 2, col 2):
Episode 1: Up=-0.100, Down=-0.100, Left=-0.109, Right=-0.500
Episode 50: Up=-0.393, Down=7.373, Left=-0.123, Right=-1.314
Episode 100: Up=0.084, Down=7.990, Left=1.434, Right=-0.881

Final learned policy (grid layout, best action per state):
Down | Righ | Down | Up
Down | [Down] | Down | [Up]
Righ | Righ | Down | [Down]
Righ | Righ | Righ | GOAL

Average cumulative reward - first 10 episodes : -26.00
Average cumulative reward - last 10 episodes : 3.40
Convergence: The rising trend (and reduced variance) in cumulative reward over
episodes shows the agent is learning a better policy - it is reaching the goal
with fewer steps and fewer obstacle penalties as Q-values converge toward their
optimal values via the Bellman update.

[Plot saved as exp2_reward_plot.png]
```

Cumulative Reward per Episode — Plot

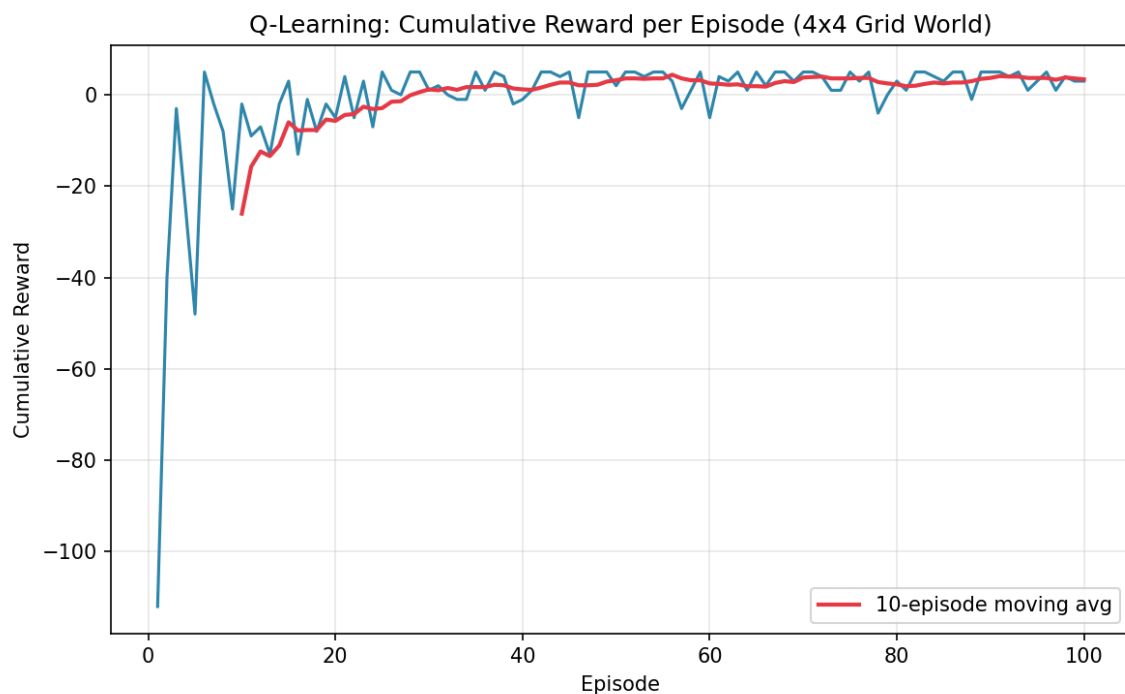


Figure: Cumulative reward per episode with 10-episode moving average, showing convergence.

Result Summary — Experiment 2

- Average cumulative reward rose from -26.00 (first 10 episodes) to $+3.40$ (last 10 episodes), showing the agent learned to reach the goal with fewer steps and fewer obstacle penalties.
- Q-values for state 0 and state 10 moved away from 0 toward higher values for actions that lead toward the goal (e.g., 'Down' at state 10 rose to ≈ 7.99 by episode 100), confirming the policy is converging via the Bellman update.
- The final learned policy routes the agent from the start (top-left) generally Down/Right toward the goal at state 15, avoiding the obstacle cells at 5, 7, and 11.

Rubric Reference (for Evaluation)

Criteria	Weightage	Level 4 – Exemplary
Understanding of Concepts	25	Thorough understanding; effectively integrates multiple concepts/technologies
Experimental Design	30	Well-planned, systematic implementation with optimal solution
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results